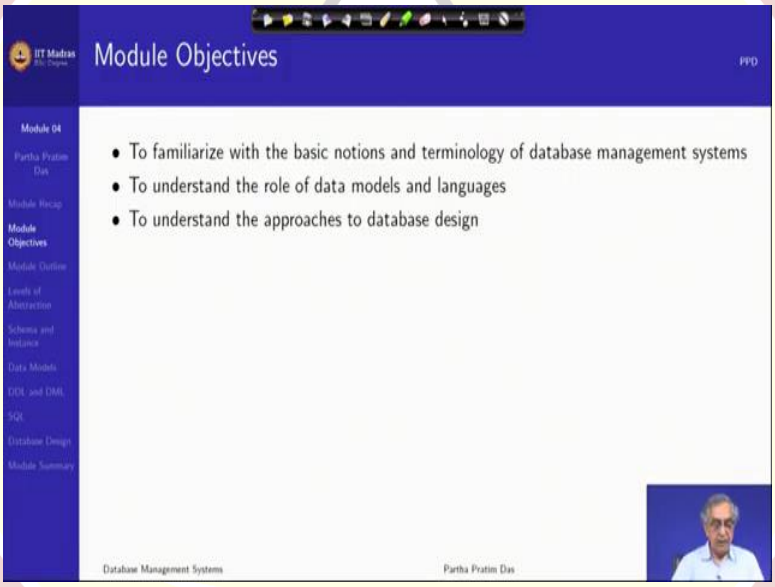
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering,
Indian Institute of Technology, Kharagpur
Introduction to DBMS/1

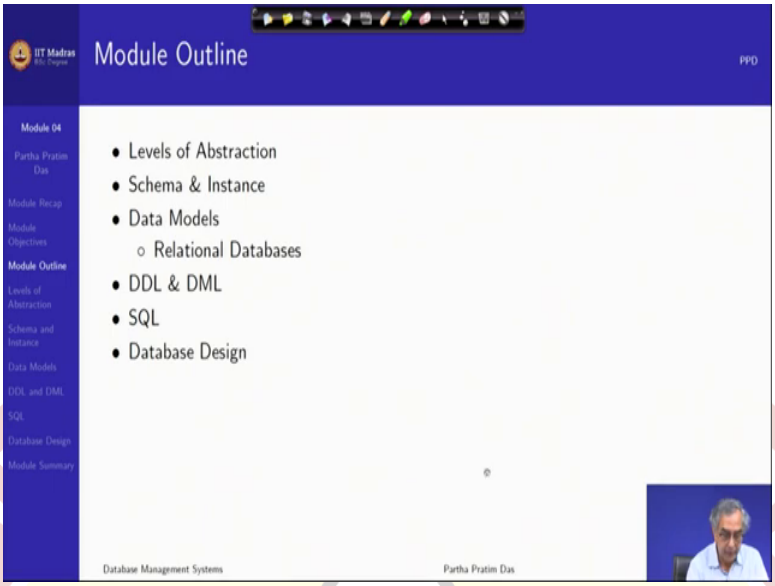
Welcome back to module 4 of Database Management Systems course in the IIT, Madras online B.Sc. program. In the earlier two modules we have discussed at length about why we need database systems and some of its basic advantages over a file-based handling solution and some of its shortcomings as well. From the module today onwards, we will start introducing the basic notions of the database system. So, I have called this module as Introduction to DBMS and this will continue in the next module as well.

(Refer Slide Time: 1:10)



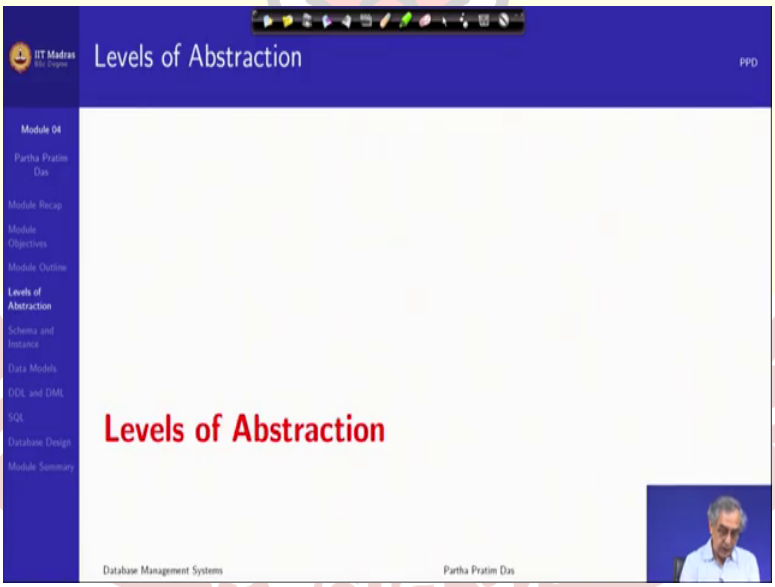
So, in this we will, this is what we have done last time comparison of file-based system and database. What we want to do now is to familiar with the basic notions and terminology of the database management system. We first need to learn the jargons that are involved and understand the basic data models and languages and the approaches to database design.

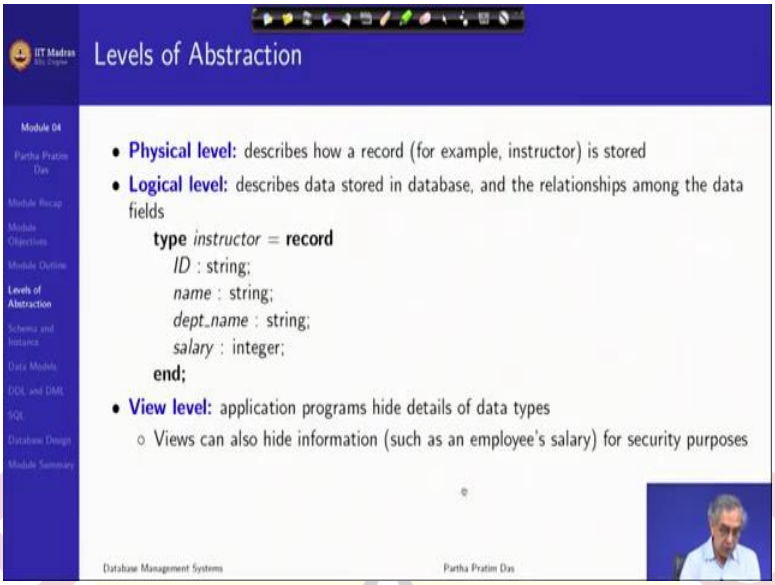
(Refer Slide Time: 1:37)

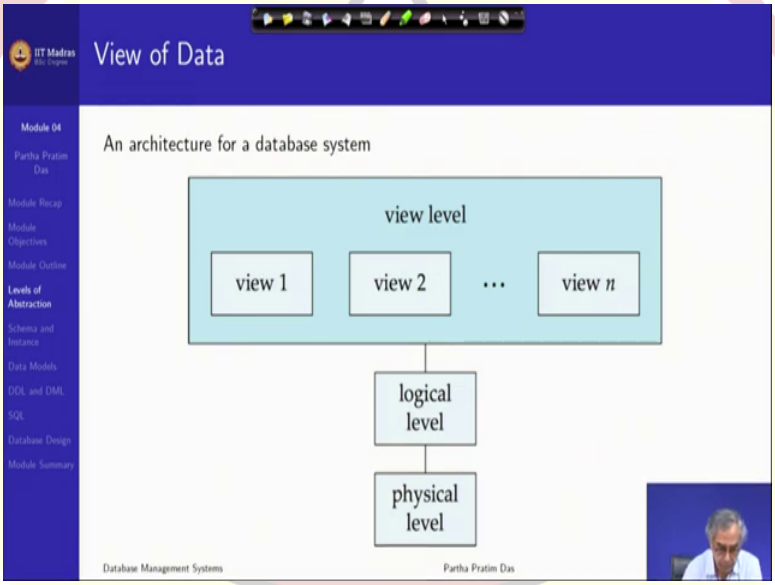


So, that is what we will talk of. In specificity we will talk about these topics, most of which you will be able to see on the left-hand banner.

(Refer Slide Time: 1:48)







So, the first thing about a database system is the levels of abstraction. What it says is that you do not look at the data just at one way; maybe that is what we often do when we do a spreadsheet, but in a database system a data is looked at multiple levels typically three levels one is called the physical level, one is called the logical level and one is called the view level.

Physical level is where you actually describe how the record is, which is stored, that is the electronic descriptor of the data, so that is a physical record. It is what actually goes into your memory or your desk, so it is called the physical one. Logical level describes data stored in the database and the relationship, so its physic, compared to physical, which is more like bits and bytes and organizations.

Logical talks about the basic abstraction of the data. For example, what is an instructor? An instructor is a record. What is a record? A record is a collection of multiple fields. What are fields? These are attributes which are of the same type and specific values can be kept. So, we saying that at a logical level we are looking at the data in this form, in a physical level it could be different blasted bits and bytes that go in.

So, we say the type instructed is a record where there is a ID, which is some kind of a string, there is name which is another kind of a string, there is a department name which is also a string, there is salary which is an integer number. So, this is the next layer over the physical layer that we get to see. Interestingly, there is a third layer which is what we actually get to see. The logical level is primarily for the programmers or the database engineers.

The view level is for the application programmers and through that several details of the data can be used, shown or hidden. For example, of the instructor, a certain view which talks about what course this instructor is instructing or which student this instructor is supervising, we would need to know the idea of the instructor, the name, the department, but not obviously the salary.

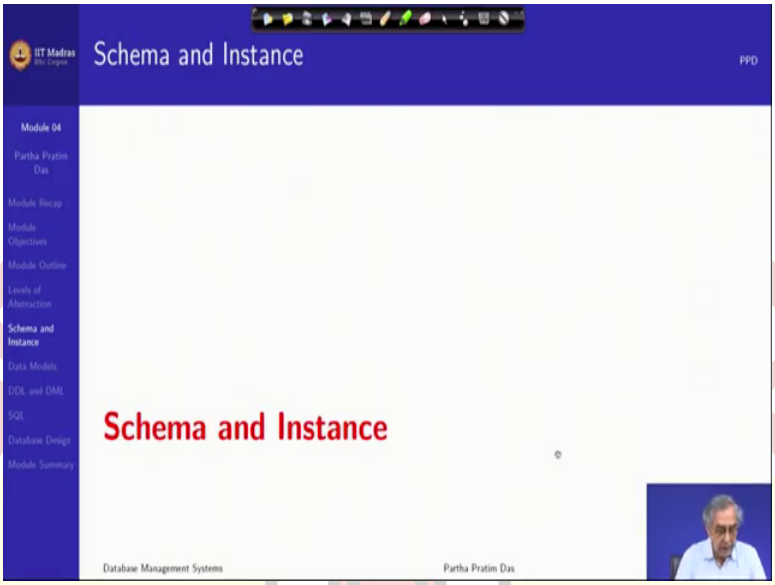
So, if you look at the logical view all four fields are available, but in the view level we will make a selective view of what is required by that application. Another very interesting point that you may keep here, for example, a view may actually show information in a derived form, not the actual information. How is it? Let us say along with this we keep the date of birth, DOB of the instructor. Now at a view we can produce what is the age of the instructor, age of the instructor is not stored, it cannot be stored because it changes every day.

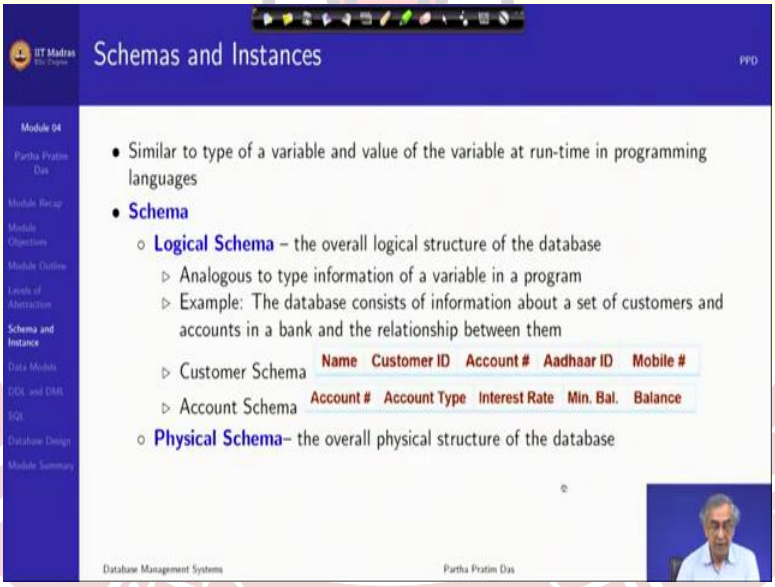
So, what you store in the logical level, what you model in the logical level is the date of birth, but what you need, you do not need to know the date of birth. You need to know whether this person is eligible for senior citizen vaccine group or for 45 to 60 vaccine group or for 18 to 45 vaccine group. You need to know whether the person has attained that age. So, that can be computed from the date of birth and given in the view, so that is the view layer.

So, three basic layers of abstraction: physical level, logical level and the view level, and of course, the physical level and logical level will be one, you cannot have multiple instances of the same instructor at the same time, but the view level could be multiple, depending on whether you are showing the involvement of the instructor in courses or in her salary

computation or in the vaccination program and so on so forth. So, this is about the levels of abstraction.

(Refer Slide Time: 7:04)



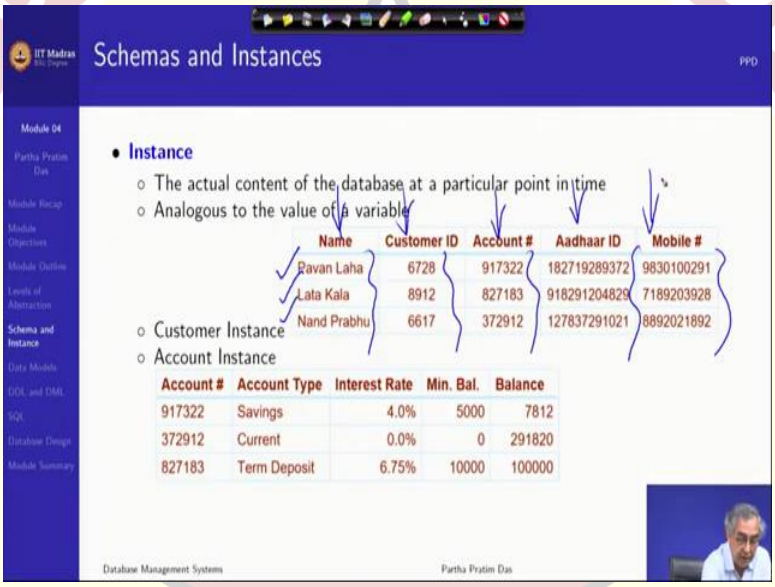


The second is the difference between schema and instance which is very-very important. In simple terms schema is the way the data will be organized and instance is the actual value of the data. So, it is very similar to the type of a variable and the value of the variable at runtime, these are the two differences. So, schema again can be logical or it could be physical. Physical schema corresponds to physical levels of abstraction. Logical schema corresponds to logical levels of abstraction, simple.

So, now, if you, for a logical schema typically you will have this kind of say a customer schema, how you describe a customer, you need to know the name, the customer ID, account number, Aadhaar ID, mobile number, so on so forth. So, that gives you the schema. That does not tell you anything specific about one customer, but it tells you how the customer will be described.

So, similarly you can have account schema; anything that you do for that you will have a schema, which is kind of the logical structure through which you can describe any entity of that group. And a physical schema is overall physical structure of the database of which we will come at a later point of time.

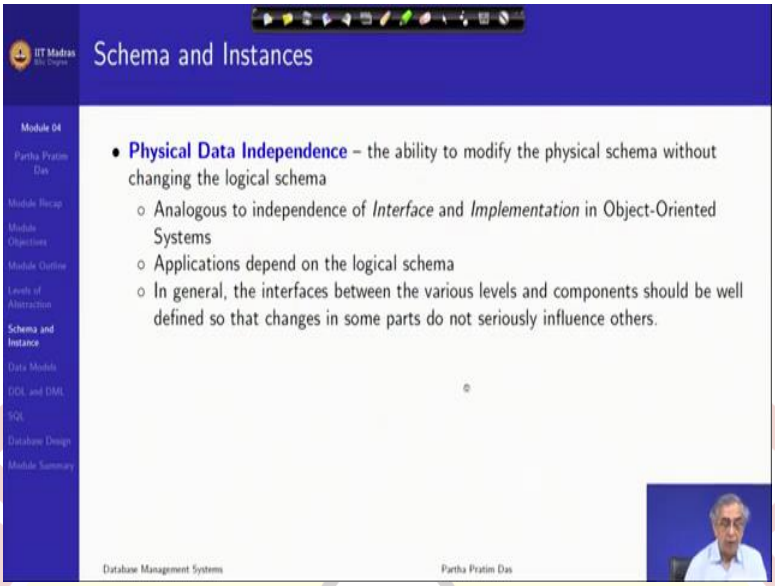
(Refer Slide Time: 8:26)



In contrast and instance is when based on a schema you have one or more records or data values available, so that is analogous to the value of a variable. So, we talked about the customer schema, here what we have is the actual customer instance. So, there are three records, each has a name, has a customer ID, account number, Aadhaar ID, mobile number, so that is an instance.

So, instances may get added, instances may get removed but the schema remains the same. Obviously if you change the schema then all instance get impacted, if you add another attribute, if you remove an attribute, these are attributes, these are all called attributes, the components of the schema. So, it is like a table, the structure of the table is a schema and the contents of the table is a instance. So, this is the basic terms that will be used.

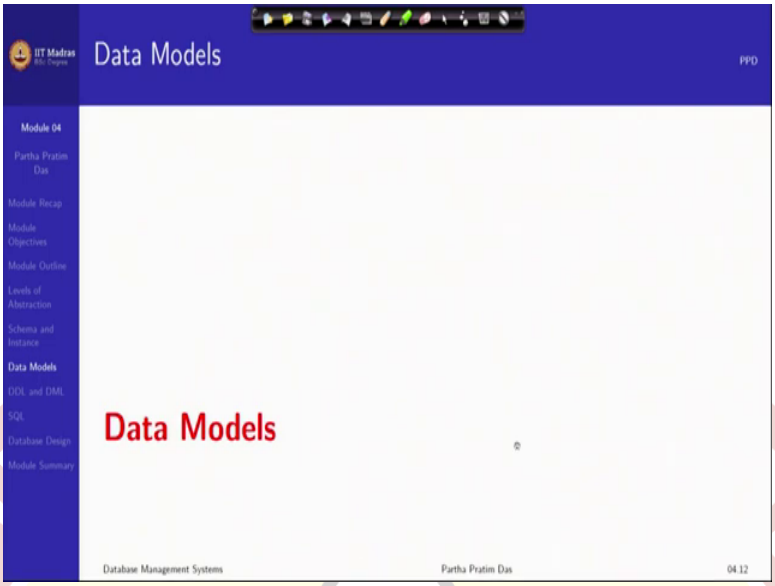
(Refer Slide Time: 9:30)

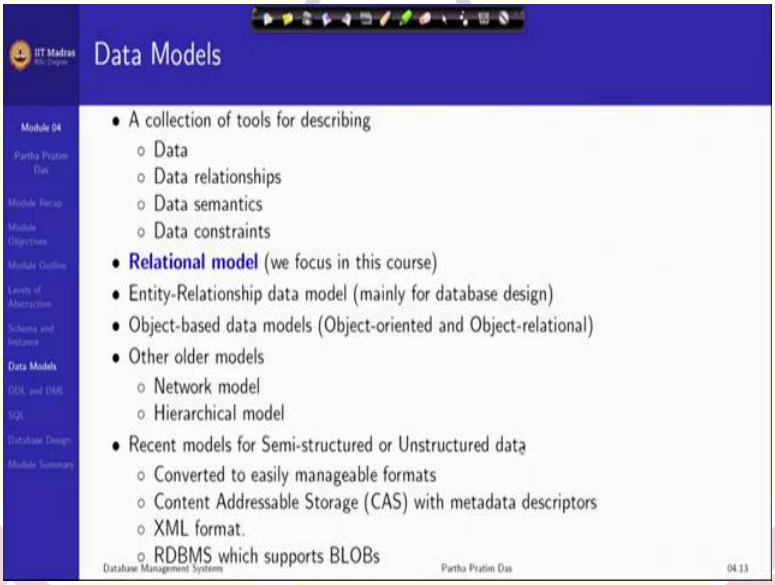


Now, in the schema and instance there is a concept of physical data dependence as to you should implement the schema or design the schema in a way so that when certain changes are made at the physical level of abstraction that should not significantly affect the logical level, so that these two can kind of remain independent. If I make some changes in the logical level it should not change my physical level, at least significantly.

So, it is like the object-oriented concept of interface and implementation. We said that the interface should be invariant because that is what the rest of the world gets to see. Implementation might change, but if that change needs the interface to be affected, then you have a problem, then you cannot work easily, so that similar kind of. So, your physical schema might change, you might migrate from one storage system to another. But your logical schema should not get affected by that significantly, so that is known as a physical data independence.

(Refer Slide Time: 10:45)



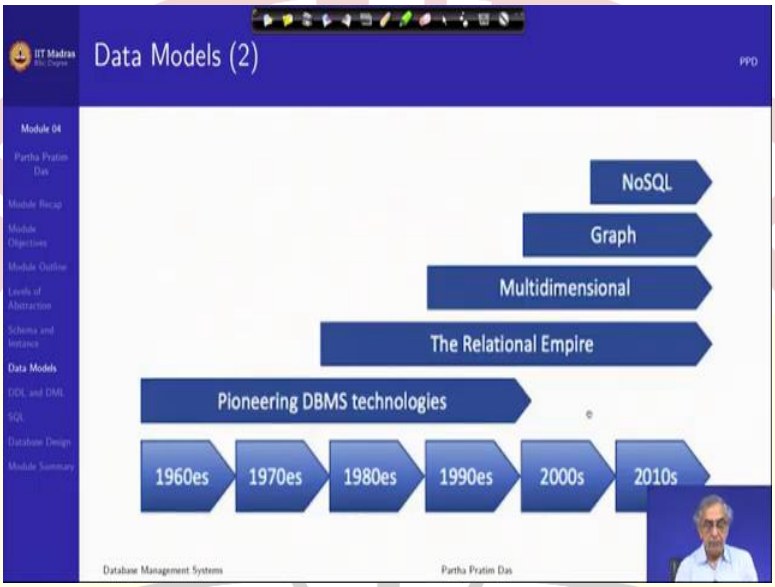


Let us get into models of data, data models. So, what is a data model? It is a collection of tool that describes the data, the relationship, the semantics that is meaning, the constraints that must happen, for example, you are doing a railway reservation, you will see that different fields will tell you what kind of different ticket quota you can get, so data will have all of that and there are several models.

What we will significantly discuss in this course is the relational model, then there is a entity relationship data model, which will be used primarily for database design, anybody who has done object oriented systems or say kind of UML will be familiar with this because equivalent models exist there.

There are objects-based data models, earlier for a long there used to be network model, hierarchical model and over the last 10–15 years there are emergence of models for semi-structured or unstructured data. We will not talk about this much right now, but those are data which are not, as I said are not like texts and numbers, but like images, videos, audios or natural language sentences and so on.

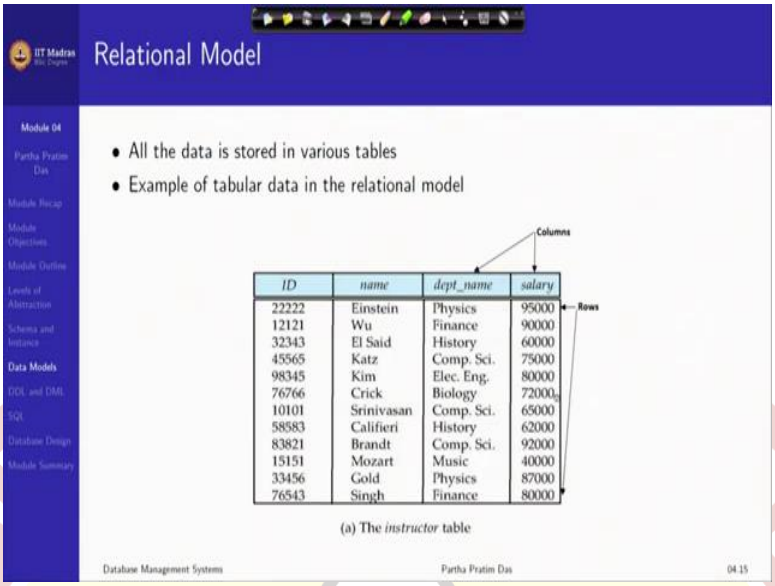
(Refer Slide Time: 12:15)



So, again reproducing the same diagram I had shown in an earlier module that this is how the models have evolved starting from the 1960s where from 60s to 90s the database technology primarily evolved to the present form and within that from 1980s relational model has been the dominant and it continues to be significantly used. But from the 90s we have multi-dimensional models taking care of multiple types of data items and we have, from 2000s we have graph model interconnections, who is whose friend and a big, big model.

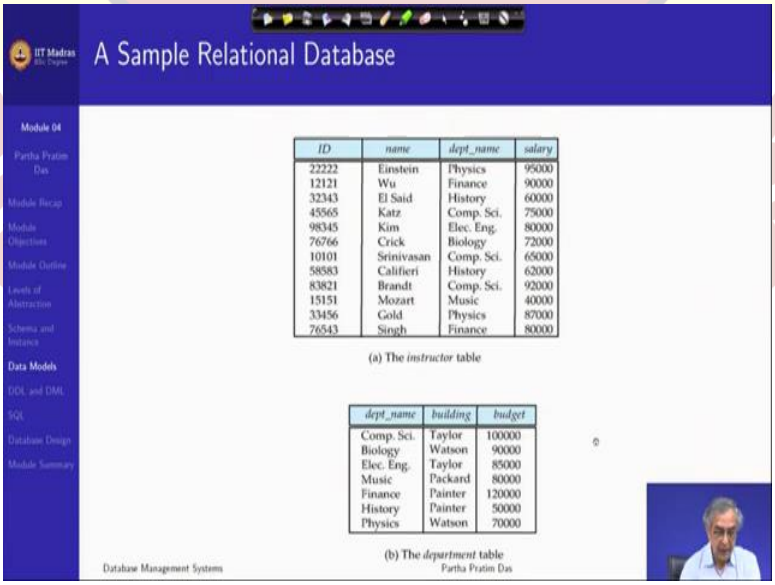
In Facebook we are always being friend, tagging, tagging, tagging, friend, friend post tag, everything is a graph, relationship, who is in my friend, who is not in my friend, who is in my group, who is not in my group, who the people tagged on this particular post, who other people who are not tagged, were tagged but cannot, but must be able to see and so on so forth. So, that is a whole lot of graph data and you need modeling for that and of late the last decade what we have is the consequent NoSQL models.

(Refer Slide Time: 13:31)



But our mainstay our main food will be relational model where everything is stored in terms of table, which are called relations, we will come to that mathematics later on, but very simply there are columns which are called attributes and they have particular name which tells you the schema and there are rows which are records, which are the values. So, you can very easily see very intuitive and very amenable to certain kinds of algebra called the relational algebra, which we will talk about later on.

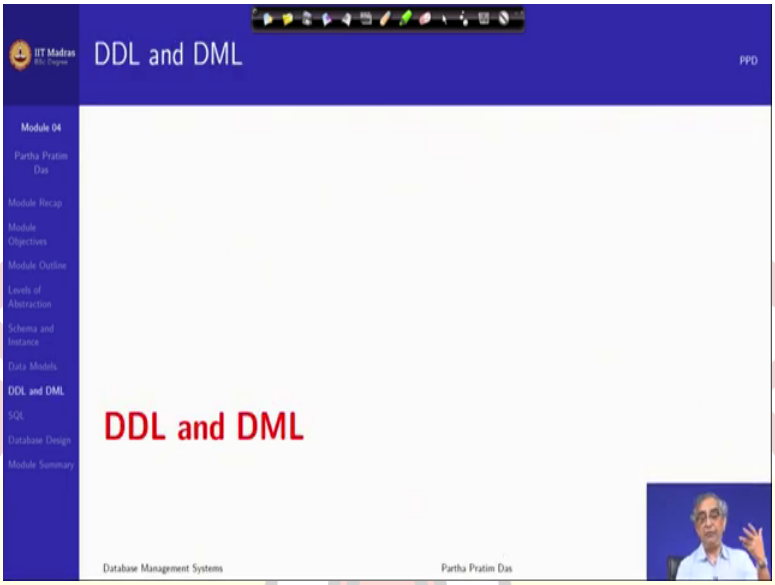
(Refer Slide Time: 14:04)

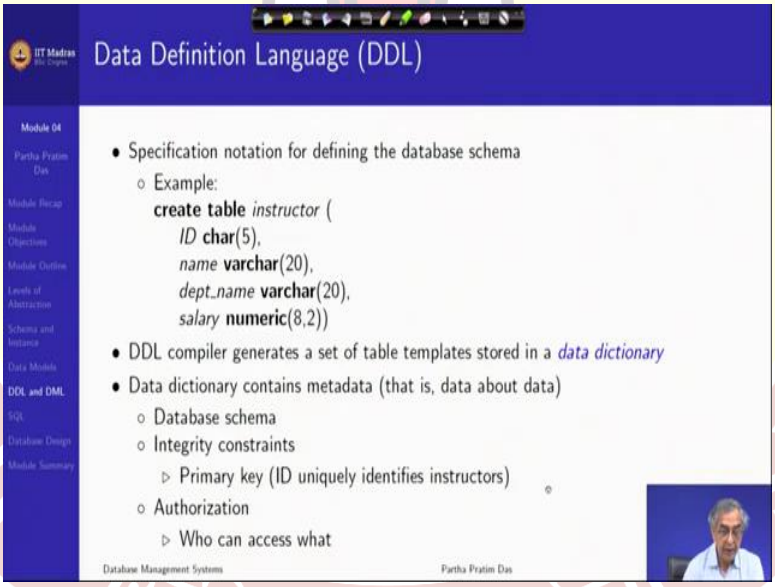


So, these are different examples of tables, the instructor table, the department table, so you can see that for the instructor table there is ID, name, department name, and salary, the example, in

the department table we have department name, building, budget and so on, so these are the different relational databases and we will have lot of designs on that.

(Refer Slide Time: 14:27)



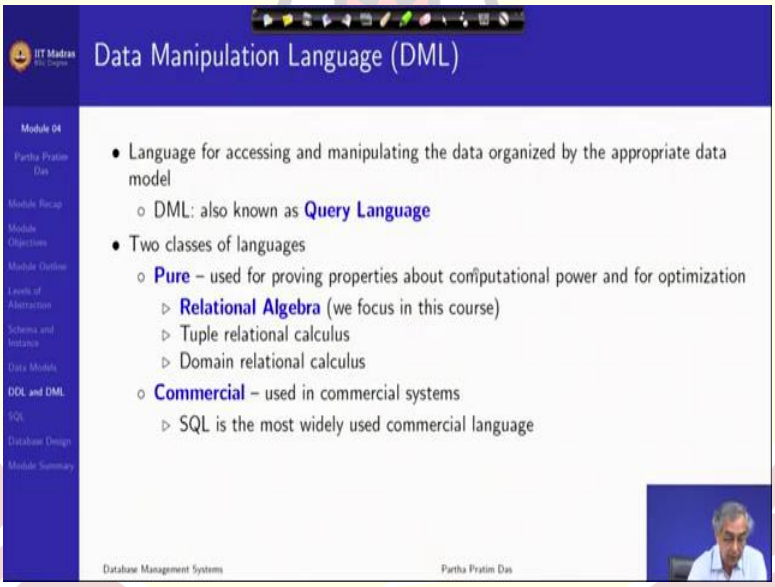


The next is DDL and DML. DDL stands for Data Definition Language. So, what you have got is you have a schema an instance, so you need a way to express a schema; you need an way to manipulate with the instance. The way to deal with the schema is DDL - Data Definition Language because it defines the structure of the data, it defines the schematic of the data. So, when we talked about type instructor is equal to record and so on, that structure can be created using this kind of a create table as you are seeing here.

You can see these fields; and it will create a corresponding table schematically of the structure. So, that is a basic DDL Data Definition Language component and DDL compiler sets a table template in a data dictionary. This is a very interesting concept that in a database all schema information is also kept in terms of tables because you will also need to know that this table, instructor, has this fields, the first field is character and will be of five characters.

How will you remember that? So, you make further tables in the database system itself which keeps track of that information and that is what is known as a data dictionary, which keeps track of, so if you refer to a particular table, you immediately from the data dictionary know where the table is, what are the attributes, what are their types, so if you want to do certain data manipulation all that or make some changes to the table, the dictionary will have to come into the play.

(Refer Slide Time: 16:19)



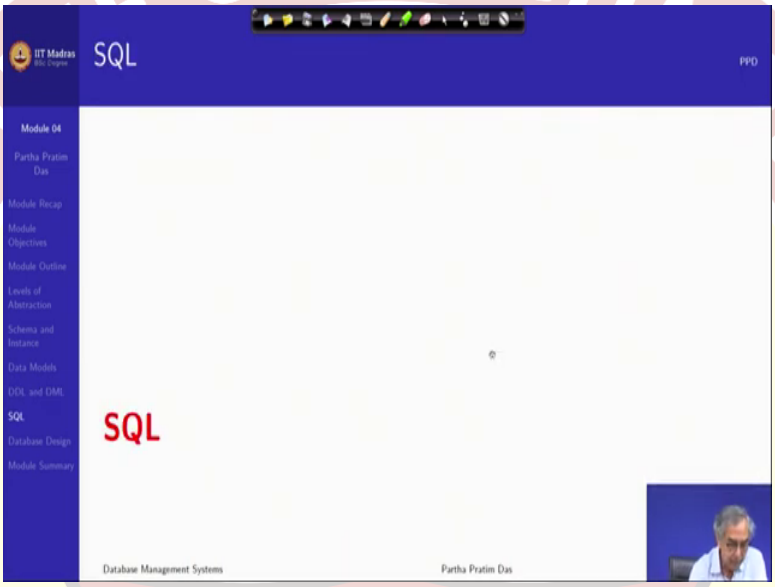
Then we have the other part, which is called the Data Manipulation Language, which basically plays around with the schema. It is often popularly also called as query language. So, by this what you saw in terms of that banking transaction sample comparison example you saw select from where that is getting some information from table, update, that is changing some value in a table, insert, that is putting a new record in the table these are all data manipulation language examples.

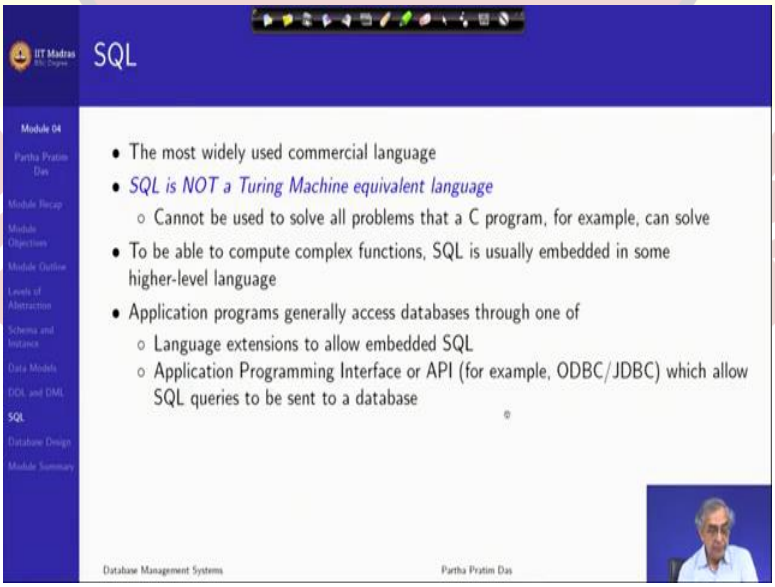
And for data manipulation we deal with specific models of or specific mathematics for the relational model. And there are three forms of this mathematics which are equivalent in a way, the relational algebra, the tuple relational calculus and the domain relational calculus, of that I

will mention about all, but I will talk primarily about the relational algebra, because that is easier in a way to understand at the beginning.

Also, this can be expressed in pure language terms I said in terms of a commercial system which is the correspondence of this is the language SQL - Structured Query Language, as the name says you can understand, but SQL is not just a DML, it is a DML as well as a DDL as well as something more, as we unfold, we will see more and more of that.

(Refer Slide Time: 18:01)





So, SQL most widely used commercial language for databases. If you are little oriented towards theoretical computer science, I would like to tell you it is not of great consequence, if you do not understand do not worry about it; is that SQL is not a Turing equivalent language, it is not

Turing complete, which means that all programs that you can write in C you cannot write it in SQL, theoretically at least anything that you write in SQL you can also write it in C. The other way it is true, because C is Turing complete.

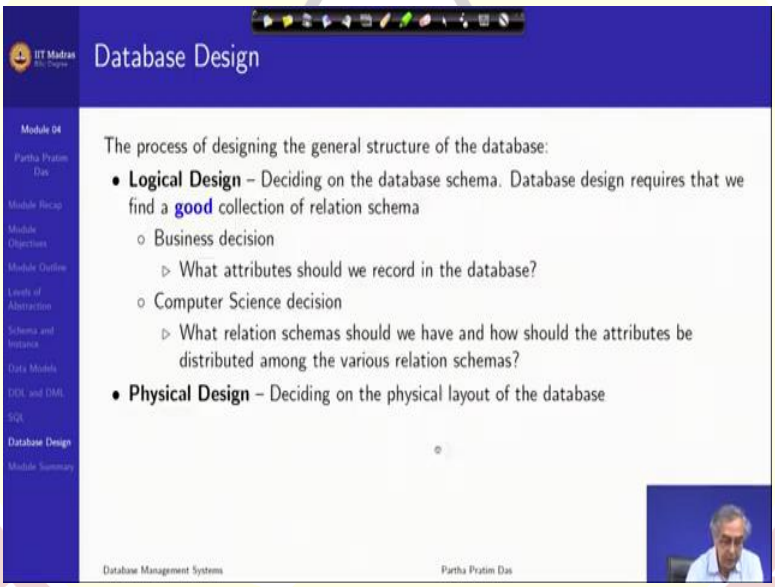
So, any program that can be written can be written in C, can be written in Python, can be written in Pascal, can be written in Java, but SQL is not that kind of a language. Here there are few problems for which you will not be able to write a program, so it is not a Turing complete language. If you have understood it is fine, if have not just ignore. But to be able to compute complex functions SQL usually is embedded used with another language.

So, you see that the whole computation may have lot of things. It is not only updating data, inserting data, checking data and so on, it may also have very detailed computation for what are the tax labs and according to the tax lab what is the percentage, how is the rebate, a big logic that goes in to actually compute your tax liability or your claim for refund and so on. In a real application you need the entire thing.

But it does not mean the fact that we have moved from file handling using a standard programming language, two database, does not mean that we will do everything in SQL itself. That is not again a very realistic approach. So, what we do we use SQL for the database and use some other language which we typically call a host language.

It could be C, it could be C plus plus, it could be Java, it could be Python, to actually do the more complex computation and interactions, so that is what we will talk of. And actually, you have a course on application development in your program I saw, I am sure that course also will elucidate lot more about this particular point that how you use SQL and database interfaces to develop appropriate applications.

(Refer Slide Time: 20:42)

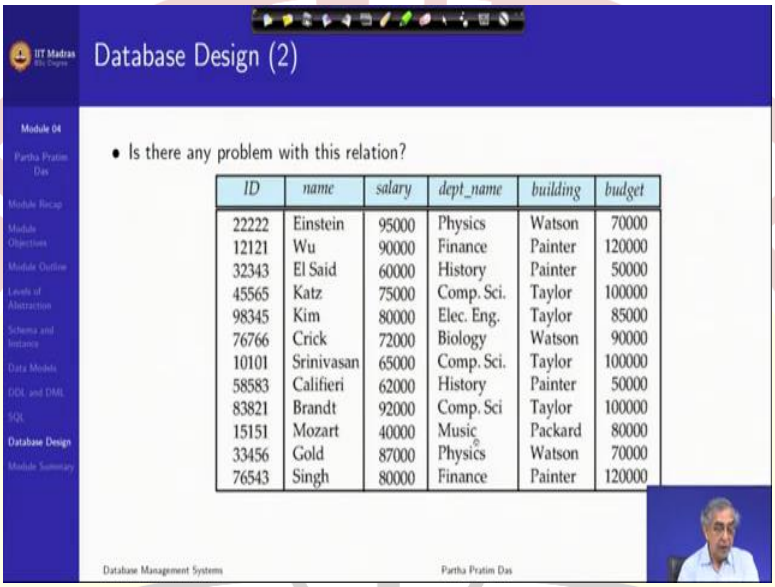


Now, database design is a complex problem. Again, corresponding to the logical view and the physical view you have a logical design and a physical design. Physical design naturally talks about the physical layout, going down to B trees and so on so forth and a logical design decides on the good schema. What is a good schema? Are any, is it enough to just find out some schema to keep the information, some form of tables will it do?

If it does not then how to choose, what is the good collection of relational schema? Related question, can there be a best design or there could be a collection of equivalent good designs and so on so forth? So, things that impact the logical designer business decisions on one side as to what I am trying to achieve, what does a business want to achieve, what we want to keep in the database, what we want to use?

And other is the computer science decisions or algorithmic decisions as to what and how it is easy to manipulate this data. How to put it appropriately in the relational schemas and so on? So, a database design is a complex process involving all these questions and therefore, you will see that we will spend a significant amount of time on the model and the associated database design issues.

(Refer Slide Time: 22:27)

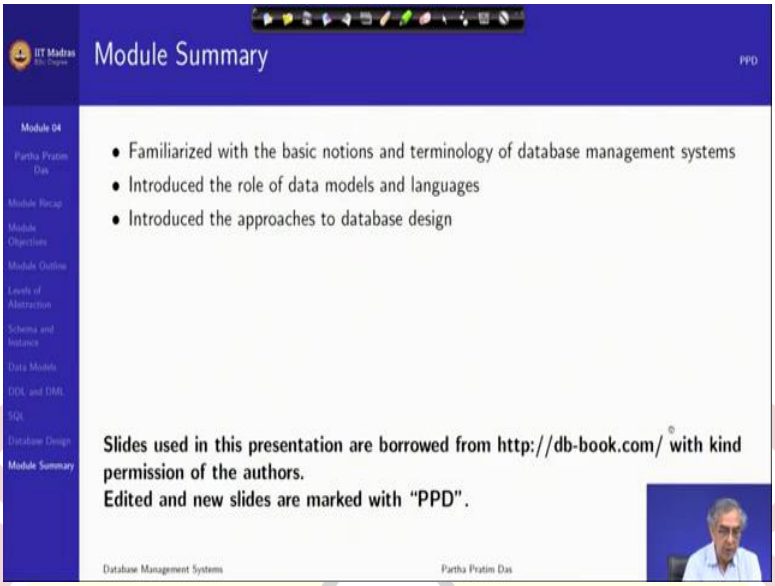


ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

This is a relation which has extension of the instructor table, instructor table had ID, name, salary and department name and department table had department name, building and budget. What we have done? We have put all this together, instead of having two tables we have made into one table. The question is, is it good? If it is good then why did we show separate tables, if it is not good why is it not good?

I will not answer that right now, I leave this as a question to you, look at the data carefully in this table and work out to yourself as to whether this is a good design, an appropriate design or something is not right in this whole thing and something else need to be done, which will give us the clue as to how to move to the basics of design.

(Refer Slide Time: 23:42)



So, with this I close on this module. So, we have familiarized with the basic notion and terminology of Database Management Systems and introduced the very basic data models and languages and just raised certain questions I should say and approaches to database design. So, as we close, we will continue on this same theme in the next module as well and talk about the remaining introductory components in module five. Till then see you, thank you very much for your attention and have a good time.